Spoken Tutorial – HNSW Implementations and Vector Databases

Title: HNSW Implementations and Vector Databases

Module	Vector Databases and Embeddings
Episode	1. HNSW Implementations and Vector Database Tools
Learning Objective	At the end of this tutorial learner will be able to 1. Identify different types of vector database tools, and set up FAISS for hands-on practice.
Approx. Duration	3-4 min
Outline	• Introduction to Vector Databases • Types of Vector Database Tools • In-Memory Libraries: FAISS • Open-Source Databases • Managed Services • Why We Use FAISS • Setting up Python Environment • Installing FAISS • Verifying FAISS Installation
Meta Tags	Vector Database, HNSW, FAISS, Chroma, Weaviate, Qdrant, Pinecone, In-Memory, Open-Source, Managed Services, AI, LLM, Python
Pre-requisite Tutorial	Basic understanding of the HNSW algorithm, and Python programming.

Script

Visual Cue	Narration
Title slide with a visual representation of vectors and a database icon.	Welcome to this Spoken Tutorial on HNSW Implementations and Vector Databases.
Learning objectives slide with bullet points and a lightbulb icon.	In this tutorial, you will learn to identify different types of vector database tools. You will also set up FAISS for hands-on practice.
System requirements slide showing icons for Linux, Firefox, Python, and pip.	To record this tutorial, I am using Ubuntu Linux OS v22.04. I am also using Firefox web browser v126.0. Python v3.10.12 is installed. And pip v22.0.2 is also installed.
Prerequisites slide with icons representing knowledge and programming.	To follow this tutorial, learners should have a basic understanding of the HNSW algorithm. They should also know Python programming.

Illustration of a vector database as an 'address book' with vectors flowing in and out, connected by HNSW graph.	We have learned about the science of HNSW. Now, let us explore the tools that implement it. We store our meaning coordinates, or vectors, in a Vector Database. Think of it as a highly optimized address book for vectors. When you get a new query, the database converts it into a vector. It then uses its internal HNSW algorithm. This instantly finds the closest vectors, which are our data chunks.
Decision tree or flowchart showing three paths for choosing a vector library.	You have three main choices for your vector library tool. Let us look at each type.
Icon for a Python script with a fast-forward symbol, representing FAISS.	First, we have In-Memory Libraries, like FAISS. FAISS is a code library from Meta. It runs directly within your Python script. Its main advantage is blazing speed. It is perfect for research, prototyping, and smaller projects. It can handle up to a few million vectors. However, it is not a full database. It does not handle data persistence, APIs, or complex filtering well.
Icons representing Chroma, Weaviate, and Qdrant around a server rack icon.	Next, consider Open-Source Databases. Examples include Chroma, Weaviate, and Qdrant. These are full, standalone database programs. You run them on a server. They offer the best of both worlds. They are powerful and can handle billions of vectors. They also support metadata filtering, and are free. The main drawback is you must manage, scale, and maintain the server yourself.
Cloud icon with a dollar sign, representing managed services like Pinecone.	Finally, we have Managed Services, like Pinecone. These are serverless databases in the cloud. You pay a provider to handle everything. The pros include infinite scalability and zero maintenance. They are very easy to use. The cons are that they can be expensive. Also, your data lives on someone else's server.
Illustration of FAISS logo with HNSW graph parameters (d, M, efSearch) highlighted.	For our hands-on work, we will start with FAISS. This allows you to directly interact with HNSW parameters. These include 'd' for vector dimension. 'M' for the number of neighbors per node. And 'efSearch' for the search-time accuracy and speed trade-off.

Screenshot of a terminal showing commands for 'mkdir', 'cd', 'python3 -m venv venv', and 'source venv/bin/activate' with '(venv)' prefix visible.	Let us set up our Python environment. Step 1: Open your terminal or command prompt. Step 2: Create a new directory for this project. Type mkdir vector_db_tutorial and press Enter. Step 3: Navigate into the new directory. Type cd vector_db_tutorial and press Enter. Step 4: Create a virtual environment. Type python3 -m venv venv and press Enter. Step 5: Activate the virtual environment. Type source venv/bin/activate for Linux and macOS. For Windows, type .\venv\Scripts\activate. Then press Enter. You should see (venv) at the start of your prompt.
Screenshot of a terminal showing 'pip install faiss-cpu numpy' and 'pip show faiss-cpu' commands and their outputs.	Now, let us install the FAISS library. Step 1: Ensure your virtual environment is active. Step 2: Install faiss-cpu using pip. Type pip install faiss-cpu numpy and press Enter. Step 3: Wait for the installation to complete. Step 4: Verify the installation. Type pip show faiss-cpu and press Enter. You should see details about the installed package.
Screenshot of a terminal showing the Python interpreter, 'import faiss' command, and 'exit()' command.	Let us quickly verify that FAISS is working. Step 1: Open a Python interpreter in your terminal. Type python and press Enter. Step 2: Try to import faiss. Type import faiss and press Enter. If no error appears, FAISS is correctly installed. Step 3: Exit the Python interpreter. Type exit() and press Enter. Pause the video and try this yourself.
Summary slide with a checkmark icon.	With this, we come to the end of this tutorial. Let us summarise.
Summary slide with bullet points summarizing the actions performed.	In this tutorial, we identified different types of vector database tools. We then set up our Python environment. Finally, we installed and verified FAISS for hands-on practice.
Assignment slide with a pencil and notebook icon.	As an assignment, please do the following.
Assignment slide showing terminal commands for deactivating venv, creating new venv, and installing faiss-gpu. Include a GPU icon.	Step 1: Deactivate your current virtual environment. Type deactivate and press Enter. Step 2: Create a new virtual environment in a different directory. Step 3: Install faiss-gpu in this new environment. Note: This requires a compatible GPU and drivers. Step 4: Compare the installation process. Also compare dependencies of faiss-cpu and faiss-gpu.

Thank you slide with EduPyramids, SINE, and IIT Bombay logos.	This Spoken Tutorial comes from EduPyramids Educational Services Private Limited. It also comes from SINE. And from IIT Bombay. Thank you for joining!